Nivel de Aplicación

Protocolos de Comunicaciones para IoT

Sergio Herrería Alonso

Curso 2026/2027

Máster Universitario en IoT

Contenido

1. Arquitecturas

2. HTTP
    - WebSockets
    - Webhooks
    - OPC UA

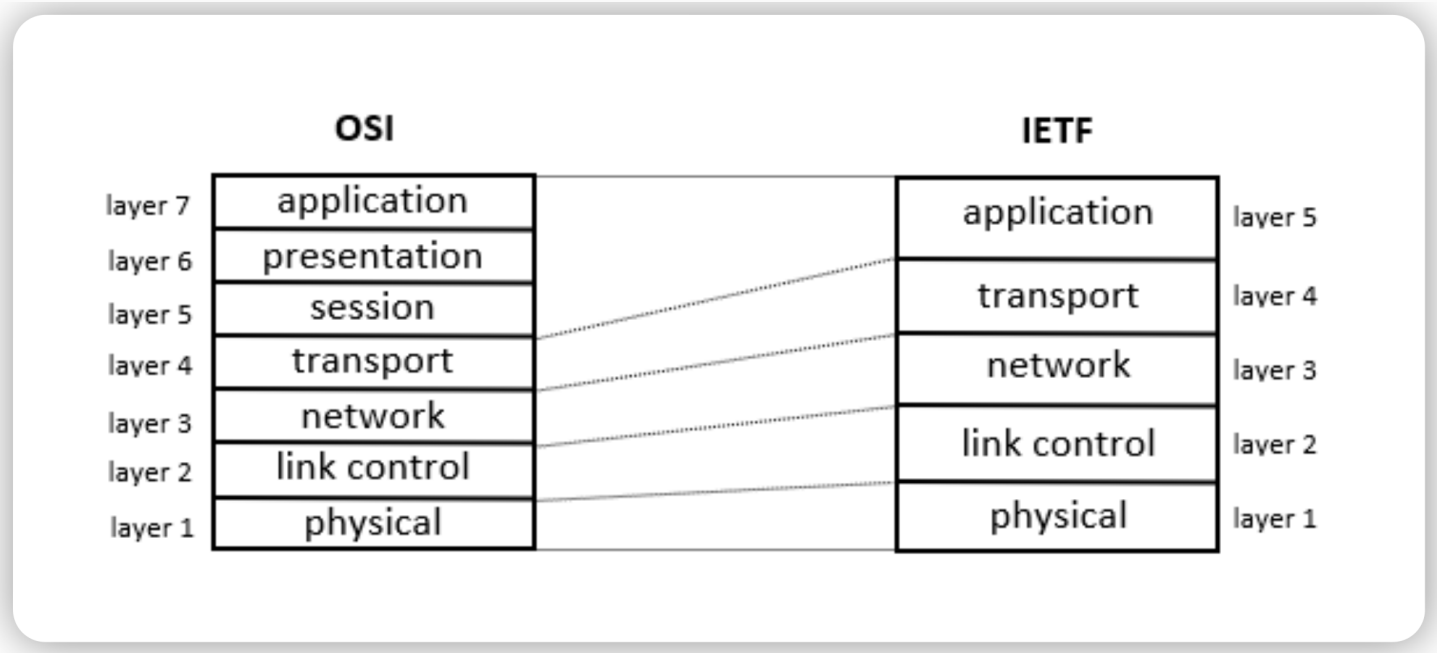
3. CoAP

4. MQTT

Arquitecturas

Aplicaciones de red

Aplicaciones distribuidas que se ejecutan en varios sistemas finales (*hosts*, servidores, dispositivos IoT...) y que requieren de una infraestructura de red



Arquitectura de red

- Capa física
  - Responsable de “mover” los bits de un nodo a otro
  - Dependiente del medio de transmisión: fibra, coaxial, radio...
- Capa de enlace
  - Responsable de hacer llegar la trama al siguiente nodo
  - Ethernet, WiFi, Bluetooth, NFC, IEEE 802.15.4, LoRa, NB-IoT, LTE...
- Capa de red
  - Responsable de hacer llegar los datagramas al nodo destino
  - IPv4, IPv6 + Protocolos de encaminamiento
  - **Capa de adaptación** (opcional, entre red y enlace): compresión de cabeceras y fragmentación de datagramas (Ej.: 6LoWPAN)
- Capa de transporte
  - Responsable de entregar los datos a la aplicación destino
  - TCP, UDP, QUIC...

La capa de aplicación

Responsable de definir cómo se intercambian la información los distintos equipos implicados en una aplicación

- Cómo se identifican los recursos y dispositivos
- Cómo se representan los datos (capa de presentación)
  - Formatos de intercambio de datos: XML, JSON, SOAP...
- Cómo se solicitan y entregan los datos
  - Formato y orden de los mensajes intercambiados
  - Gestión de sesiones (capa de sesión)
- Qué modelo de comunicación se utiliza
  - Modelos cliente/servidor y publicación/suscripción
- HTTP, CoAP, MQTT...

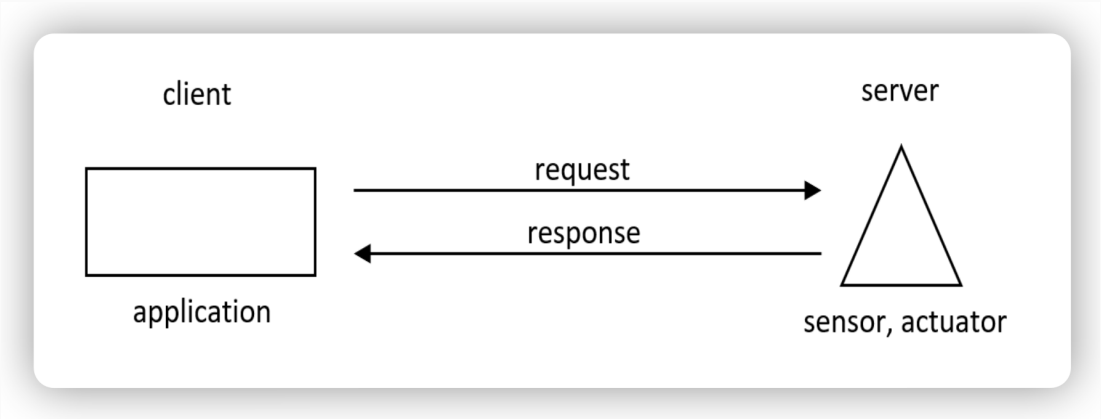
Formato de intercambio de datos: XML

```
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca>
   <libro id="bk101">
       <titulo>El Señor de los Anillos</titulo>
       <autor>J.R.R. Tolkien</autor>
       <fecha>1954</fecha>
   </libro>
   <libro id="bk102">
       <titulo>1984</titulo>
       <autor>George Orwell</autor>
       <fecha>1949</fecha>
   </libro>
</biblioteca>
```

Formato de intercambio de datos: JSON

```
{
  "biblioteca": [
     {
        "id": "bk101",
        "titulo": "El Señor de los Anillos",
        "autor": "J.R.R. Tolkien",
        "fecha": "1954"
     },{
        "id": "bk102",
        "titulo": "1984",
        "autor": "George Orwell",
        "fecha": "1949"
     }
  ]
}
```

Modelo cliente/servidor



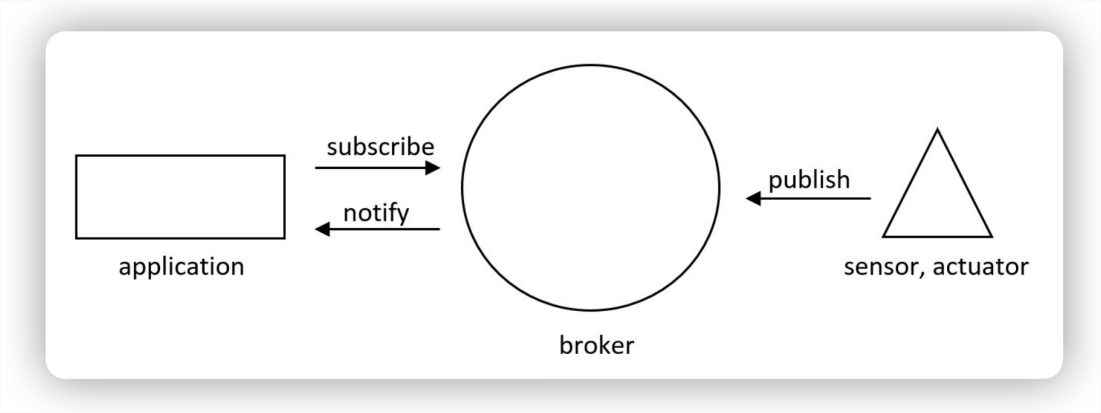
- Modelo petición/respuesta
  - La aplicación (cliente) solicita datos o envía comandos
  - El dispositivo IoT (servidor) responde enviando los datos solicitados o asentimientos
  - Para cada petición se genera una respuesta
- Comunicación síncrona iniciada por el cliente
- HTTP, CoAP, XMPP

Arquitectura REST (*REpresentational State Transfer*)

Principios de diseño para la interacción cliente/servidor

- Separación clara de la funcionalidad del cliente y del servidor, que se comunicarán a través de una interfaz bien definida, pudiendo evolucionar independientemente
- Comunicación sin estado: cada petición debe contener toda la información necesaria para que el servidor la procese
- Uso de cachés: las respuestas pueden almacenarse en la caché de clientes/*proxies* reduciendo el consumo de recursos de red
- Arquitectura en capas jerárquicas con interfaces definidas pero sin una implementación formalmente especificada
- Código bajo demanda (opcional): el servidor puede extender la funcionalidad de los clientes mediante el envío de *scripts*

Modelo publicación/suscripción



- El intercambio de datos se realiza a través del broker
  - El dispositivo IoT (publicador) envía al *broker* datos que están asociados a un *topic* determinado
  - La aplicación (suscriptor) se suscribe a los *topics* deseados
  - El *broker* notifica la recepción de nuevos datos a todas las aplicaciones suscritas al *topic* correspondiente
- MQTT, AMQP

Modelo publicación/suscripción

- Comunicación asíncrona
  - El *broker* almacena temporalmente los datos publicados y, en algún momento, los hará llegar a los suscriptores
- Operaciones
  - *advertise*: el publicador anuncia un nuevo *topic*
  - *unadvertise*: el publicador revoca un *topic*
  - *subscribe*: la aplicación se suscribe a un *topic*
  - *unsubscribe*: la aplicación revoca su suscripción a un *topic*
  - *publish*: el publicador publica nuevos datos en un *topic*
  - *notify*: el broker envía los nuevos datos publicados a todos los suscriptores asociados a un *topic*
- Modelo descentralizado
  - Red de *brokers* que operan de forma coordinada
  - Evita que el *broker* se convierta en un punto único de fallo o cuello de botella

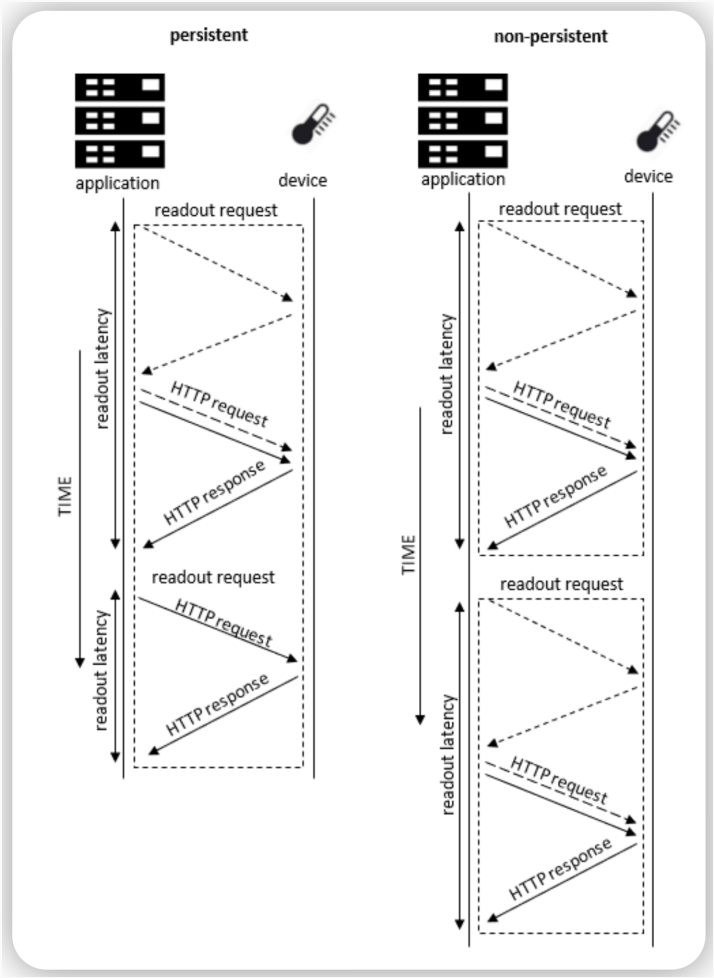
HTTP

HTTP (*HyperText Transfer Protocol*)

Protocolo para intercambiar información entre un cliente y un servidor web

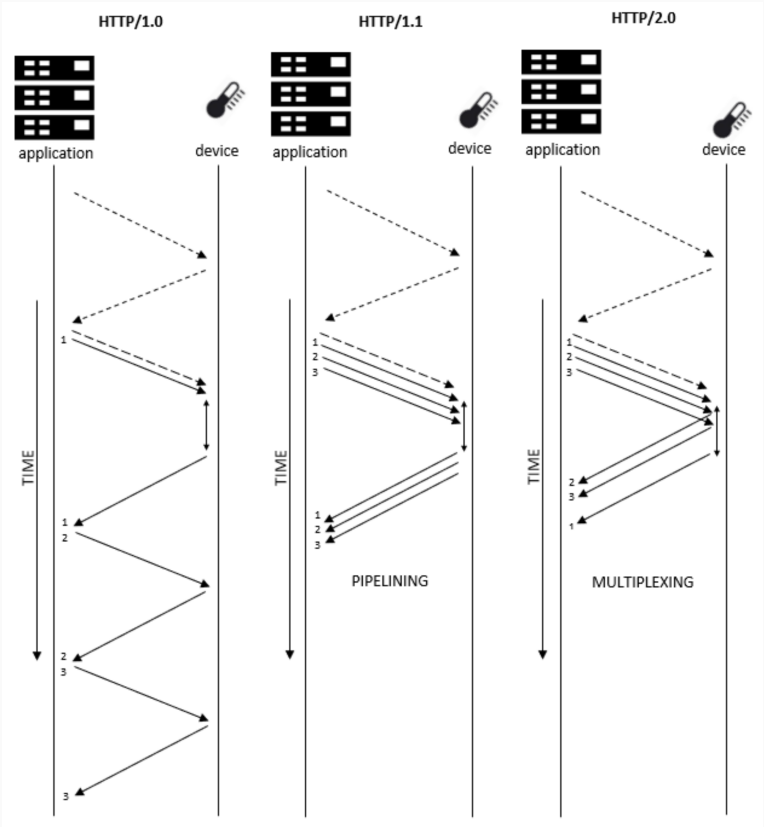
- El cliente solicita un documento al servidor
  - Habitualmente, páginas HTML, imágenes, javascript, css...
  - ...pero también medidas (sensores) o comandos (actuadores)
- Sobre TCP (puerto 80) o TLS/TCP (puerto 443)
  - También sobre QUIC/UDP (puerto 443)
- Mensajes HTTP/1.x codificados mediante caracteres ASCII
  - Muy grandes para caber en una única trama del nivel de enlace
  - HTTP/2 y HTTP/3 usan *framing* binario para optimizar el envío
- Reutilización de toda la infraestructura web disponible
  - Servidores, cachés, CDN, traspaso automático de *firewalls*...
  - Web of Things (WoT)

HTTP: Persistencia de conexiones



- Antes de enviar la petición HTTP se establece una conexión TCP
  - *Three-way handshake*:
    1. SYN (cliente)
    2. SYN/ACK (servidor)
    3. ACK (cliente)
- Tiempo de respuesta: 2·RTT
  - RTT (*Round Trip Time*)
- Conexión persistente
  - La conexión TCP se mantiene abierta para enviar múltiples peticiones/respuestas entre un mismo cliente/servidor

HTTP: Versiones



- HTTP/1.0
  - El cliente debe esperar por la respuesta a una petición antes de enviar la siguiente
- HTTP/1.1
  - *Pipelining*: el cliente puede enviar varias peticiones sin esperar a su respuesta
- HTTP/2.0
  - El servidor puede enviar las respuestas en cualquier orden, evitando el bloqueo HOL
- HTTP/3.0
  - QUIC/UDP en vez de TCP

HTTP: Mensajes

- Petición

```
GET /device101/temperature HTTP/1.1
Host: www.muiot.uvigo.es
Connection: keep-alive
User-Agent: PXO Sensor
```

- Respuesta

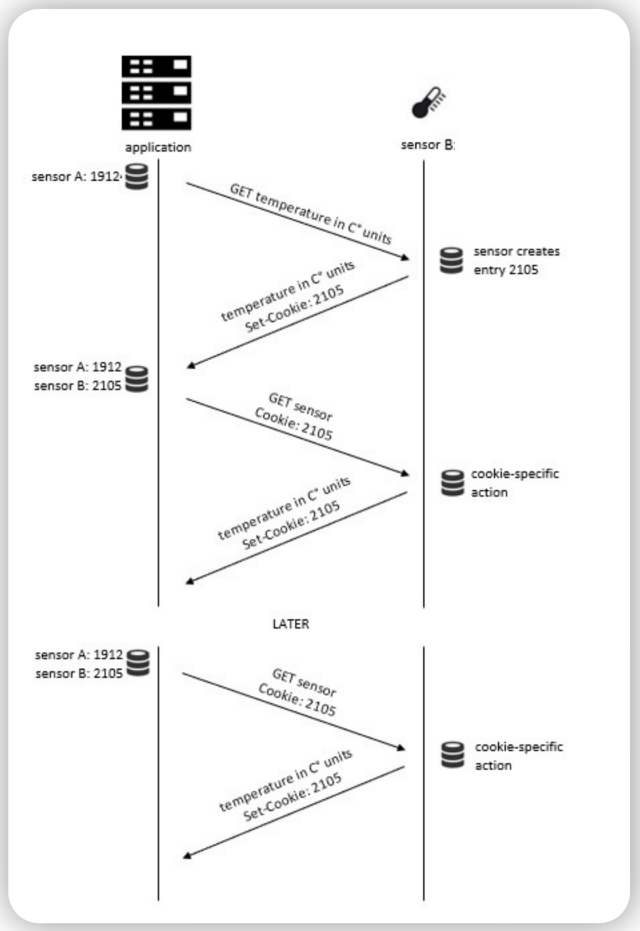
```
HTTP/1.1 200 OK
Connection: keep-alive
Date: Wed, 23 Nov 2024 08:03:12 GMT
Server: Apache/2.4.61 (Red-Hat/Linux)
Last-Modified: Wed, 23 Nov 2024 08:00:01 GMT
Content-Length: 6
Content-Type: text/plain

18.6 C
```

HTTP: Mensajes

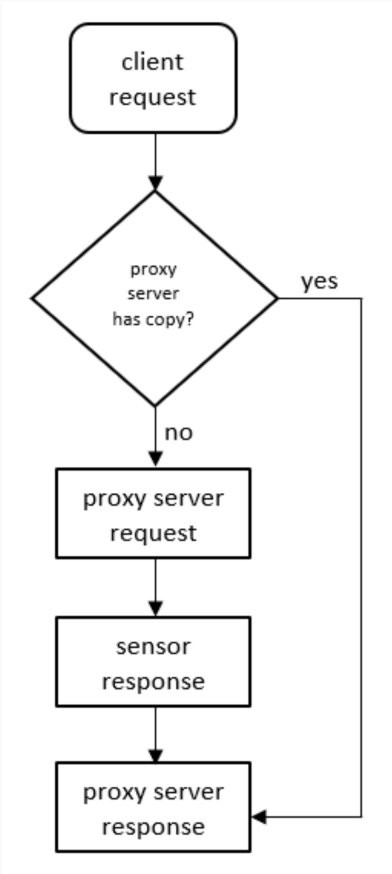
- Métodos
  - GET: obtener una medida del dispositivo
  - PUT: actualizar el estado del dispositivo
  - POST: crear un nuevo dispositivo
  - DELETE: eliminar un dispositivo
- Códigos de respuesta
  - 1*xx*: Información
  - 2*xx*: Éxito
    - 200 OK
  - 3*xx*: Redirección
    - 301 Moved Permanently, 304 Not Modified
  - 4*xx*: Error del cliente
    - 400 Bad Request, 404 Not Found
  - 5*xx*: Error del servidor
    - 505 HTTP Version Not Supported

HTTP: Cookies



- Las cookies permiten mantener información de estado durante la comunicación cliente/servidor
- Son cadenas de caracteres arbitrarias definidas por el servidor y enviadas al cliente que éste devuelve sin modificar
- Usos
  - Validación del usuario para acceder a determinados servicios
  - Personalización del aspecto/contenido según las preferencias del usuario
  - Seguimiento de la actividad de los usuarios

HTTP: Proxy servers



- Un proxy server es un servidor que hace de intermediario entre los clientes y servidores
  - Responde a las peticiones de los clientes en lugar de los servidores originales
- Mantiene en su caché una copia de los datos más recientes solicitados por los clientes
  - Devuelve inmediatamente el dato solicitado si no ha cambiado desde que se guardó en la caché
- Ventajas
  - Reducción de los tiempos de acceso a los datos
  - Reducción de la carga en los servidores originales
  - Acceso anónimo a los recursos

HTTP: Proxy servers

- Petición condicional

```
GET /device101/temperature HTTP/1.1
Host: www.muiot.uvigo.es
Connection: keep-alive
User-Agent: PXO Sensor
If-Modified-Since: Wed, 23 Nov 2024 08:03:12 GMT
```

- Respuesta

```
HTTP/1.1 304 Not Modified
Connection: keep-alive
Date: Wed, 23 Nov 2024 08:07:45 GMT
Server: Apache/2.4.61 (Red-Hat/Linux)
```

HTTP

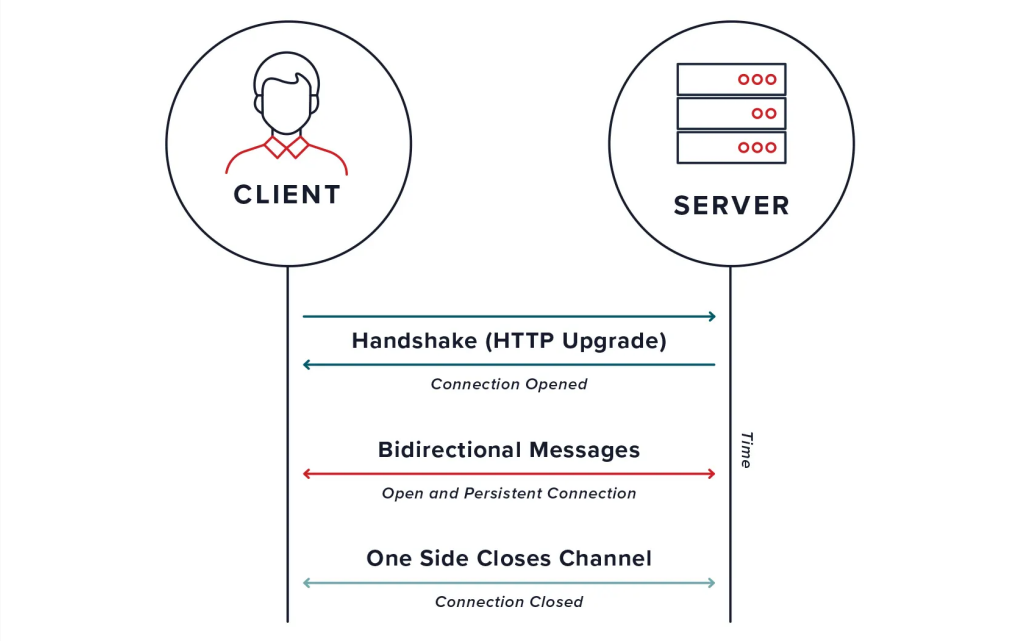
WebSockets

WebSocket

Protocolo para mantener una comunicación bidireccional interactiva entre un cliente y un servidor web

- Permite que el servidor envíe contenido al cliente sobre una conexión persistente sin que el cliente lo solicite
  - El cliente recibe datos sin tener que sondear al servidor
  - Útil para aplicaciones interactivas: chats, juegos, Google Docs...
- Protocolo estandarizado por el IETF (RFC 6455) y el WHATWG
- Requiere HTTP 1.1 (o superior) para el *handshake* inicial
- Sobre TCP (puerto 80) o TLS/TCP (puerto 443)
- Soportado por los principales navegadores y servidores web
  - Chrome, Firefox, Edge, Safari, Opera

WebSockets: Protocolo



- Requiere una fase inicial (*handshake*) sobre HTTP 1.1
  - Para cambiar de HTTP al protocolo WebSocket
- Una vez establecido el WebSocket, deja de usarse HTTP
  - El cliente y el servidor pueden enviarse datos mediante *frames* "binarios" en cualquier momento

WebSockets: Opening Handshake

- Solicitud

```
GET /ws-endpoint HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

- Respuesta

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: CTPN8jCb3BUjBjBtdjwSQCytuBo=
Sec-WebSocket-Protocol: chat
```

WebSockets: Mensajes

- Los mensajes se transmiten en *frames* con formato binario
- Mensajes de datos
  - Cadenas de texto: envío de caracteres codificados en UTF-8
  - Datos binarios: envío de vídeo, audio, imágenes...
  - Pueden enviarse fragmentados en varios *frames*
    - Útil si se desconoce a priori la longitud de los datos a enviar
    - Permite el envío de datos según van estando disponibles
- Mensajes de control
  - Ping/Pong
    - Si se recibe un mensaje Ping, debe responderse con un Pong
    - Se utilizan como mensajes *keepalive* o para verificar que el otro extremo sigue conectado
  - Close: para cerrar el WebSocket
  - No se pueden fragmentar

HTTP

Webhooks

Webhooks

Mecanismo basado en eventos para el envío automático de datos en tiempo real a través de HTTP

- Monitorizan eventos dentro de un sistema o aplicación
- Cuando ocurre el evento especificado, el *webhook* envía una solicitud HTTP POST a la URL configurada (*endpoint*)
  - Con información sobre el evento que activó el *webhook* (*payload*)
- Al igual que los WebSockets, eliminan la necesidad de sondeo
- Pero son más simples y adecuados para las comunicaciones unidireccionales
- Eficientes para notificaciones esporádicas
  - Notificación de eventos, sincronización de datos, automatización de flujos de trabajo

Webhooks: Mensajes

- Solicitud

```
POST /livestreaming-app/webhooks/donations HTTP/1.1
Host: livestreaming-app.example.com
Content-Type: application/json
Content-Length: 92

{
  "event": "donation.success",
  "data": {
    "username": "YourBiggestFan",
    "amount": 10
  }
}
```

- Tras procesar el *webhook*, la aplicación responderá con un 200 OK si la operación se realizó correctamente

HTTP

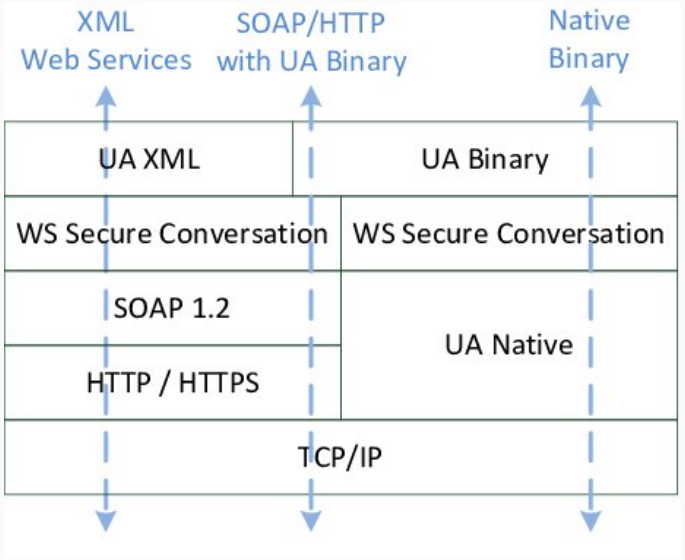
OPC UA

OPC UA (*Open Platform Communications United Architecture*)

Arquitectura cliente/servidor para el intercambio seguro de datos en entornos industriales

- Definida por la OPC Foundation para la Industria 4.0 y la IIoT
- Arquitectura orientada a servicios independiente tanto de la plataforma hardware como del sistema operativo
- Incorpora mecanismos de seguridad avanzados para cifrado, autenticación y auditoría
- Sobre TCP (puerto 4840), HTTP (80) o HTTPS (443)
- Define dos mecanismos para la codificación de los datos:
  - Codificación XML: “legible” pero ineficiente
  - Codificación binaria: eficiente pero “no legible”

OPC UA: Pila de protocolos



- Introduce una capa de seguridad a nivel de sesión
- Transmisión de mensajes XML
  - Se codifican con SOAP y luego se envían sobre HTTP/HTTPS
- Transmisión de datos binarios
  - Se envían sobre SOAP/HTTP (obsoleto) o directamente sobre UA nativo (mecanismo habitual)
  - Sobre HTTPS/WebSockets para integración con plataformas web

SOAP (*Simple Object Access Protocol*)

Estándar W3C basado en XML para intercambiar información estructurada entre distintas aplicaciones

- Interoperabilidad entre sistemas y aplicaciones heterogéneas
- La aplicación crea un mensaje SOAP en formato XML para solicitar o enviar datos
- Sobre cualquier protocolo de transporte (habitualmente, HTTP)
- Elementos de un mensaje SOAP
  - Sobre (*Envelope*): elemento raíz que identifica al mensaje SOAP
  - Encabezado (*Header*): elemento opcional con información sobre cómo debe ser procesado el mensaje
  - Cuerpo (*Body*): elemento obligatorio que contiene el mensaje
  - Fallo (*Fault*): elemento opcional con información sobre los errores producidos durante el procesado del mensaje

SOAP: Mensajes

- Solicitud

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getProductInfo xmlns="http://warehouse.example.com/ws">
      <productId>827635</productId>
    </getProductInfo>
  </soap:Body>
</soap:Envelope>
```

- Respuesta

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getProductInfoResult xmlns="http://warehouse.example.com/ws">
      <productId>827635</productId>
      <productName>Toptimate 3-Piece Set</productName>
      <price>96.50</price>
      <inStock>true</inStock>
    </getProductInfoResult>
  </soap:Body>
</soap:Envelope>
```

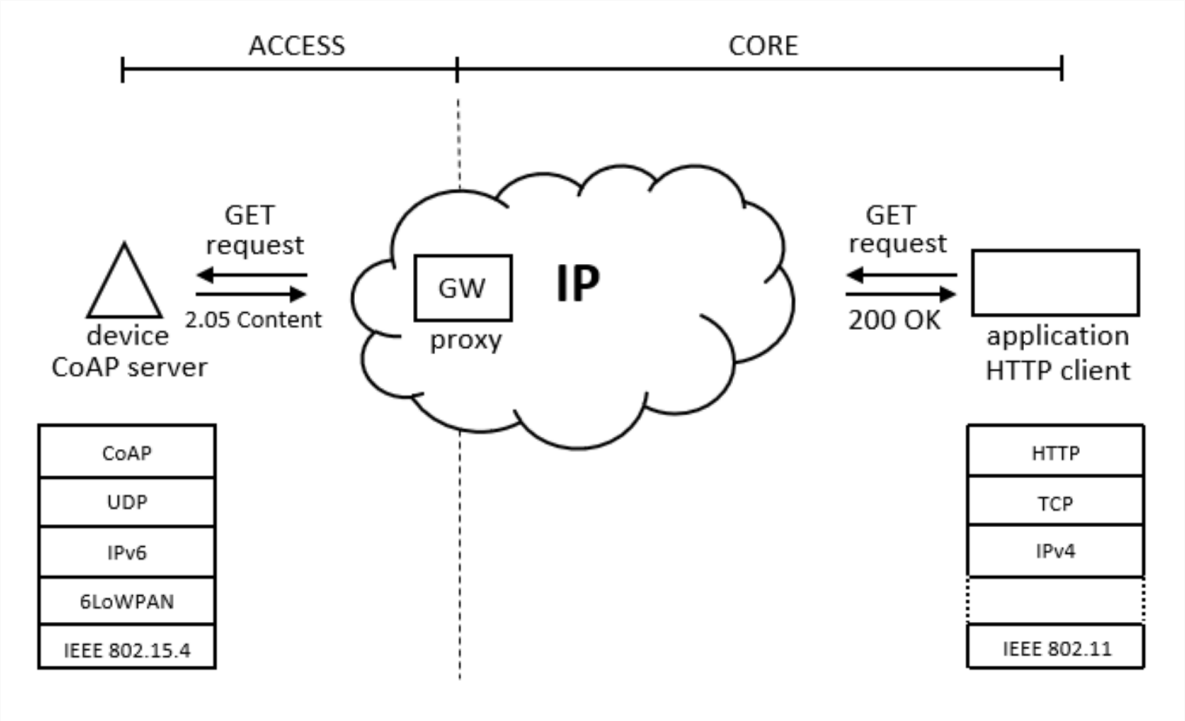
CoAP

CoAP (*Constrained Application Protocol*)

Protocolo “ligero” para el intercambio de información con dispositivos de recursos limitados

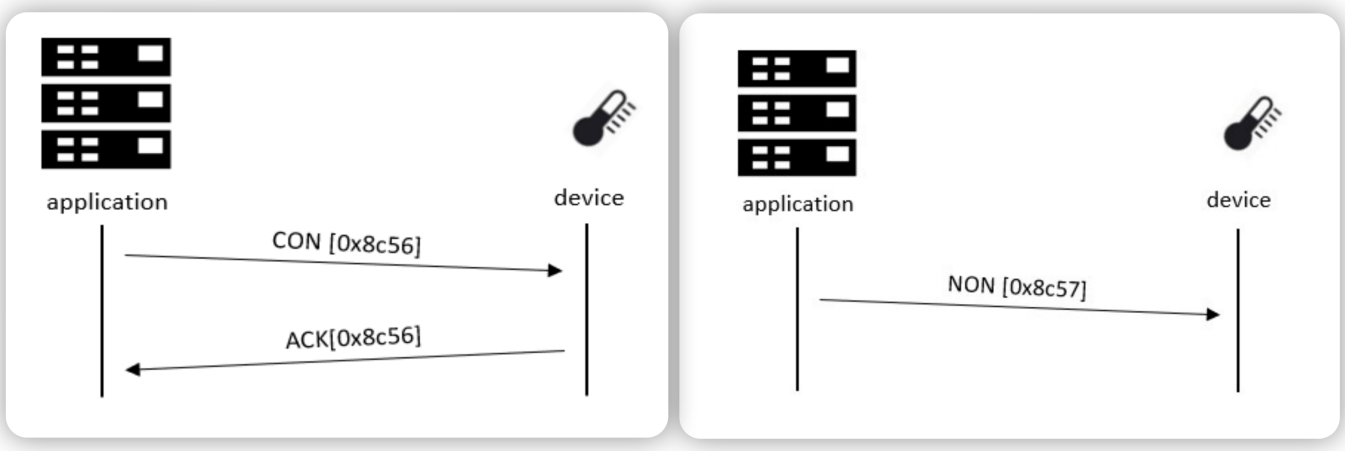
- Protocolo diseñado para acceder a dispositivos IoT
- Protocolo cliente/servidor estandarizado (RFC 7252)
- Sobre UDP (puerto 5683) o DTLS (puerto 5684)
  - Los errores y reintentos se gestionan desde la aplicación
  - También sobre TCP/TLS (RFC 8323)
- Mensajes CoAP codificados como secuencias “binarias”
  - Mensajes pequeños que caben en una única trama del nivel de enlace (ideal para enviar sobre 6LoWPAN/IEEE 802.15.4)
- Soporte multicast

CoAP: Escenario



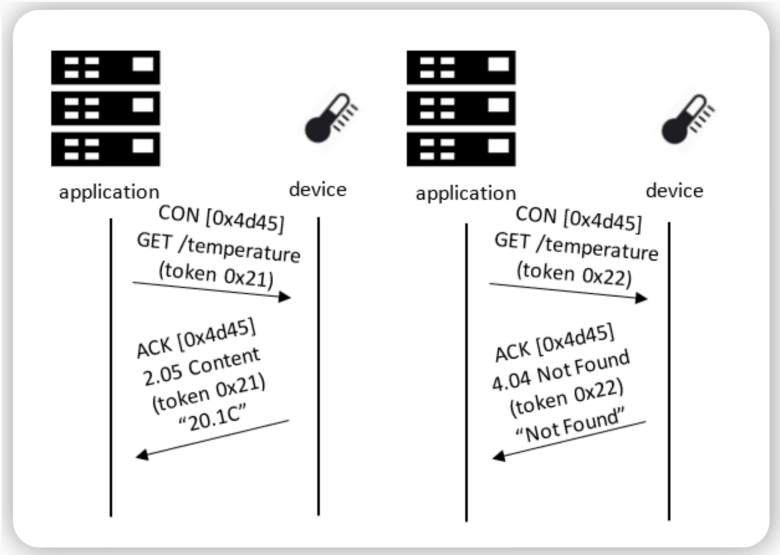
- CoAP en la red de acceso y HTTP en el núcleo
- Requiere gateway (proxy) que traduzca los mensajes de un protocolo a otro

CoAP: Modos de operación



- En el modo “confirmable” el servidor envía asentimiento
  - Si no recibe el ACK, el cliente reenvía el mensaje hasta 4 veces duplicando el tiempo entre reintentos (*exponential backoff*)
- En el modo “non-confirmable” el servidor no envía ACK
- Si el servidor es incapaz de procesar la petición, envía un mensaje RST (en ambos modos)

CoAP: Modos de operación



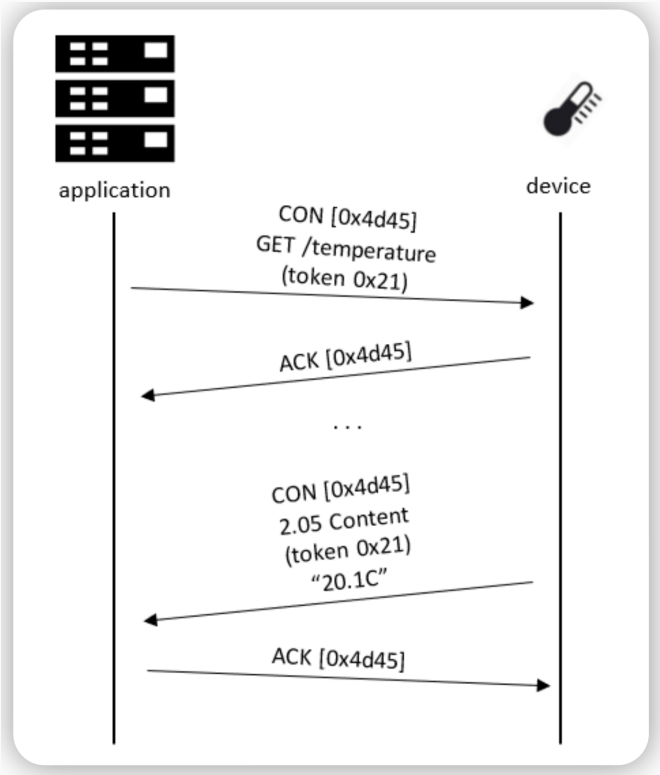
- Respuesta piggybacked
  - El ACK y la respuesta se envían en el mismo mensaje
- *Message ID*: fiabilidad, duplicados
- *Token*: petición↔respuesta
- Código de mensaje: *a.bb*

  $a = 0$: petición
  - $bb = 01$: GET, $bb = 02$: POST
  - $bb = 03$: PUT, $bb = 04$: DELETE

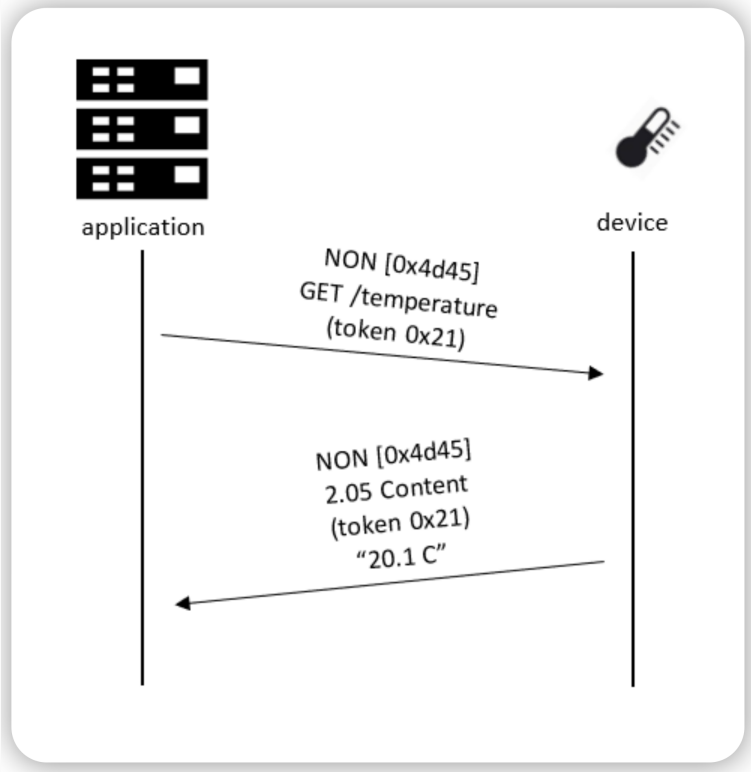
  $a \in \{2, 4, 5\}$: respuesta
  - $a = 2$: éxito
  - $a = 4$: error del cliente
  - $a = 5$: error del servidor

  $a = 7$: señalización (ping, abort...)

CoAP: Modos de operación

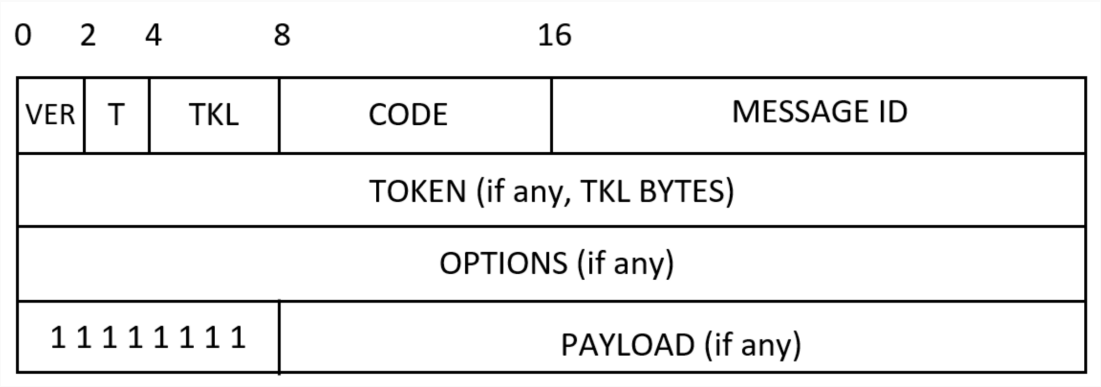


Confirmable (non-piggybacked)



Non-confirmable

CoAP: Formato de mensaje



- Versión (2 bits): 1
- Tipo de mensaje (2 bits): 0 (CON), 1 (NON), 2 (ACK), 3 (RST)
- Longitud del *token* (4 bits): 0–8 (en bytes)
- Código de mensaje (8 bits): *a.bb*, con *a* (3 bits) y *bb* (5 bits)
- Identificador del mensaje (16 bits)
- *Token* opcional (0–8 bytes)
- Secuencia de opciones en formato TLV (*Type-Length-Value*)
- Datos (*payload*)

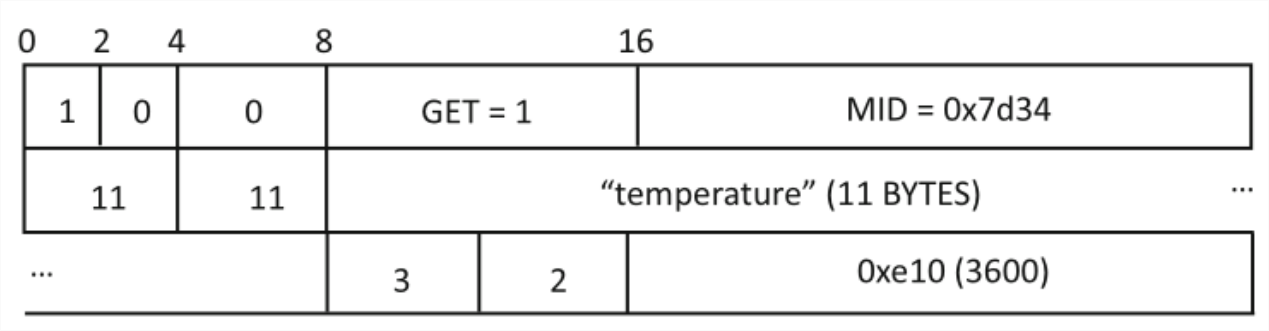
CoAP: Formato de mensaje

CoAP option types

Option name	Option type
If-Match	1
Uri-Host	3
ETag	4
If-None-Match	5
Location-Path	8
Uri-Path	11
Content-Format	12
Max-Age	14
Uri-Query	15
Accept	17
Location-Query	20
Proxy-Uri	35
Proxy-Scheme	39
Size1	60

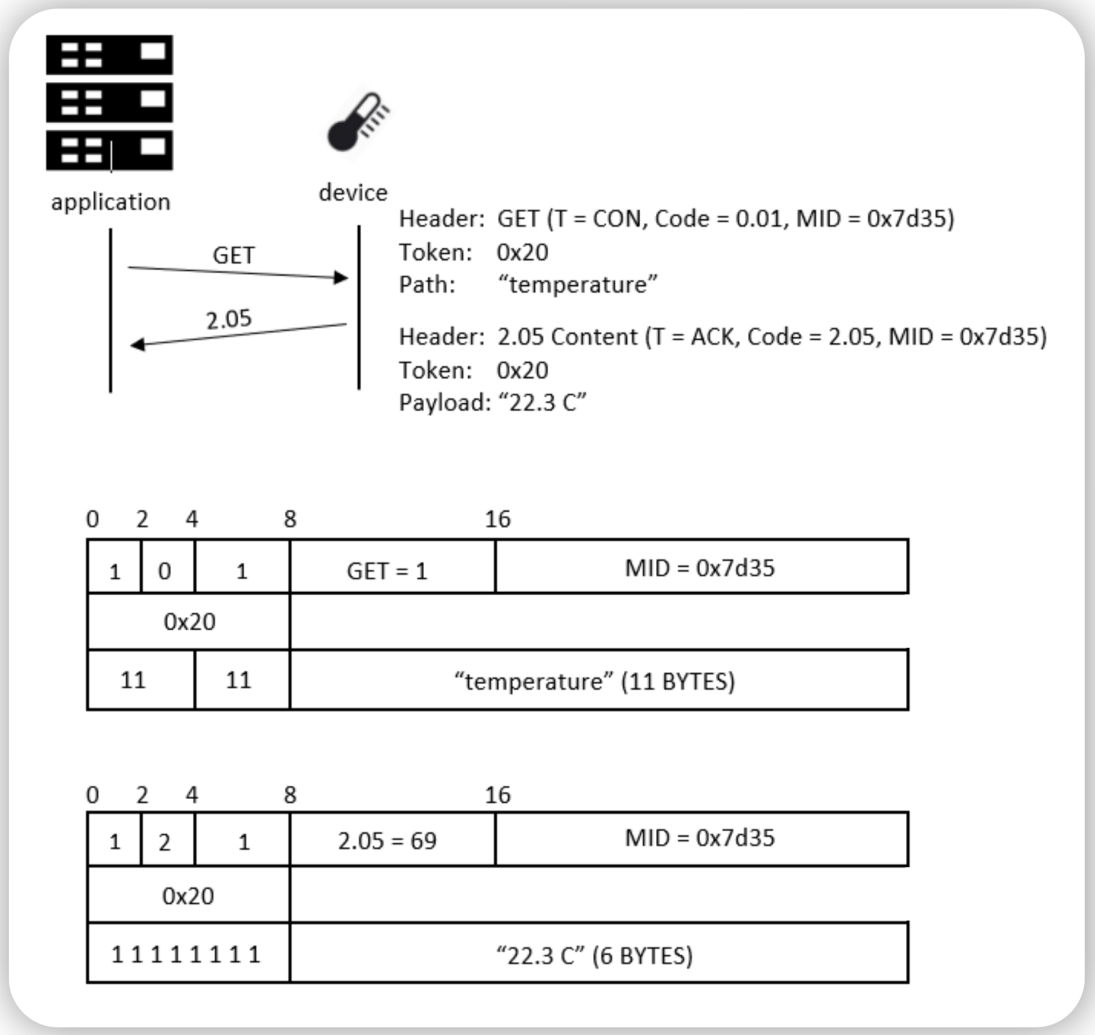
CoAP: Formato de mensaje

Ejemplo:

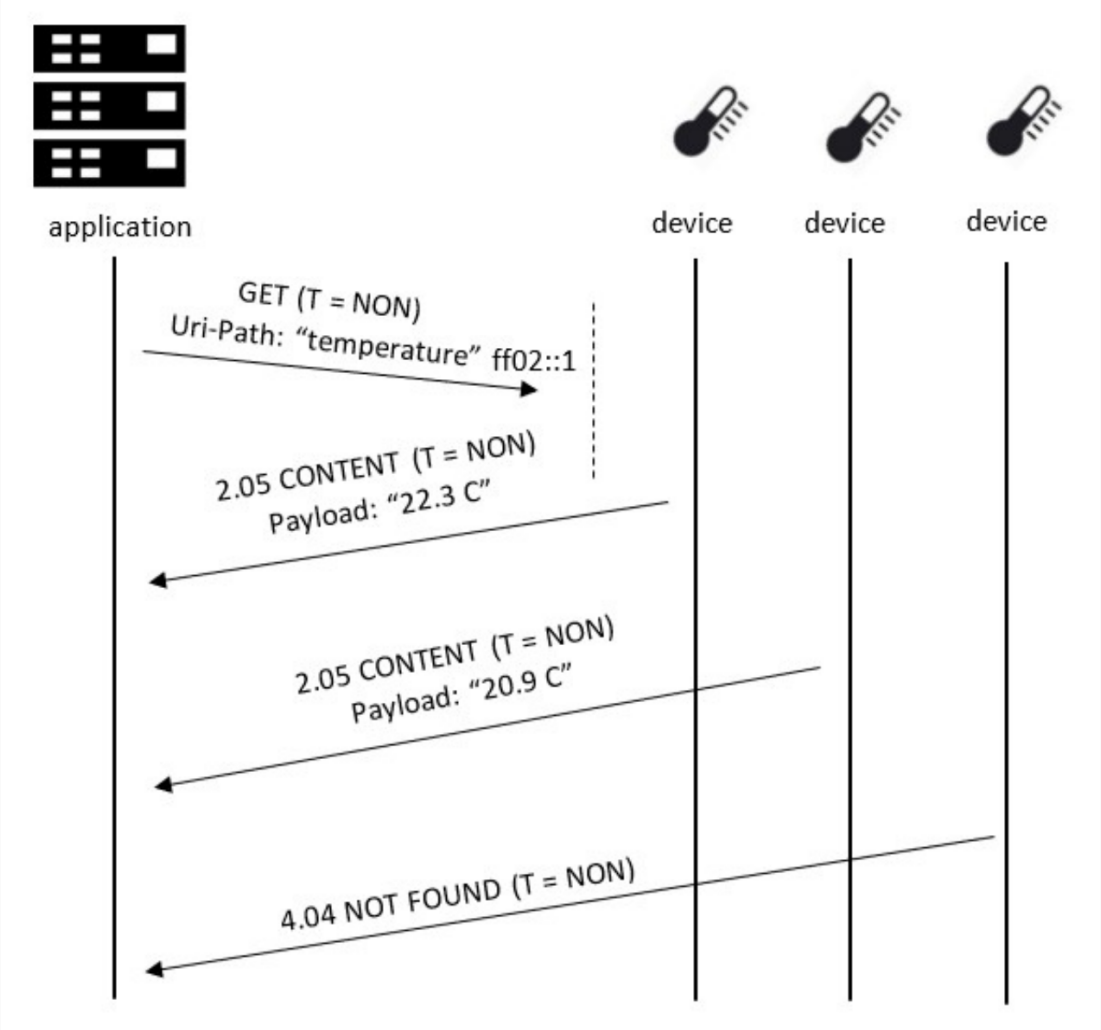


- Petición GET confirmable
- Sin *token* ni *payload*
- Opción `Uri-Path`
  - Identificador del recurso solicitado al servidor
  - tipo=11, longitud=11, valor=temperature
- Opción `Max-Age`
  - Tiempo máximo en caché de la respuesta
  - tipo=14, longitud=2, valor=3600

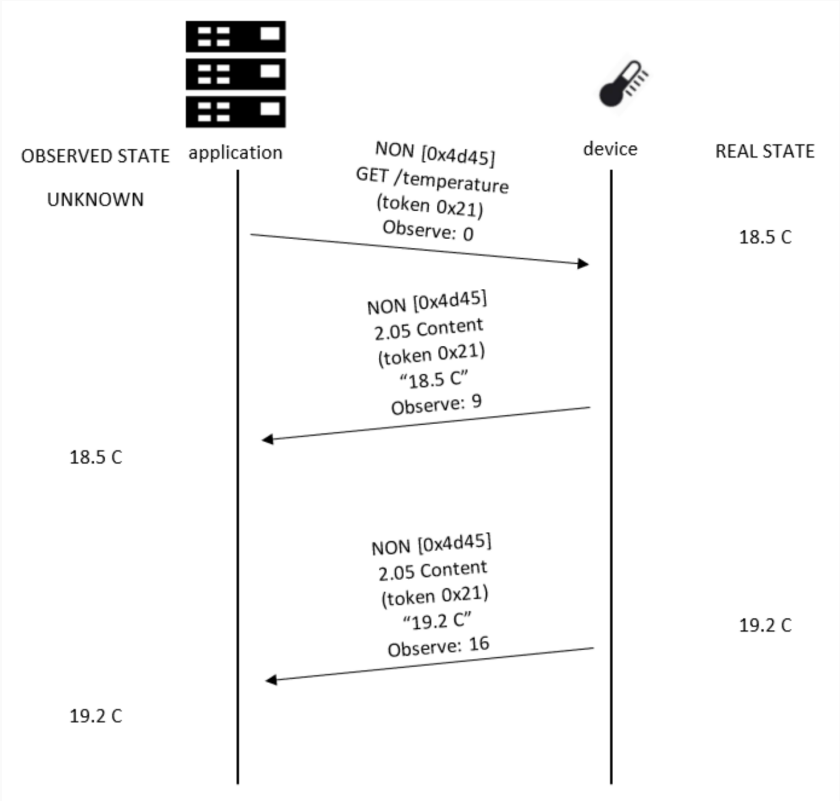
CoAP: Formato de mensaje



CoAP: Multicast



CoAP: Observaciones



- Opción observe (RFC 7641)
  - Servicio de suscripción
  - Obtención del valor actual y futuro de objetos
  - Petición con *Observe*: 0
- El servidor mantiene una lista de observadores
  - Una entrada en la lista para cada cliente/*token*
  - Envía respuestas con el valor actualizado
  - *Observe*: referencia temporal
- Para darse de baja, el cliente envía petición con *Observe*: 1

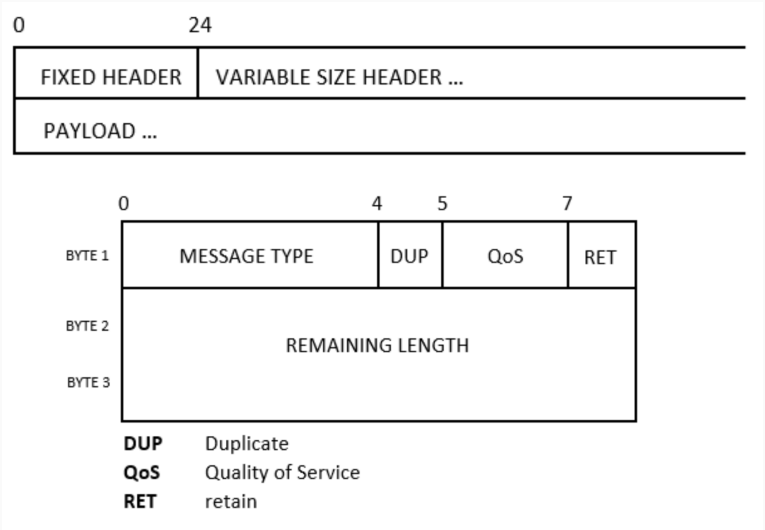
MQTT

MQTT

Protocolo de publicación/suscripción “ligero” diseñado para el intercambio de información con dispositivos IoT

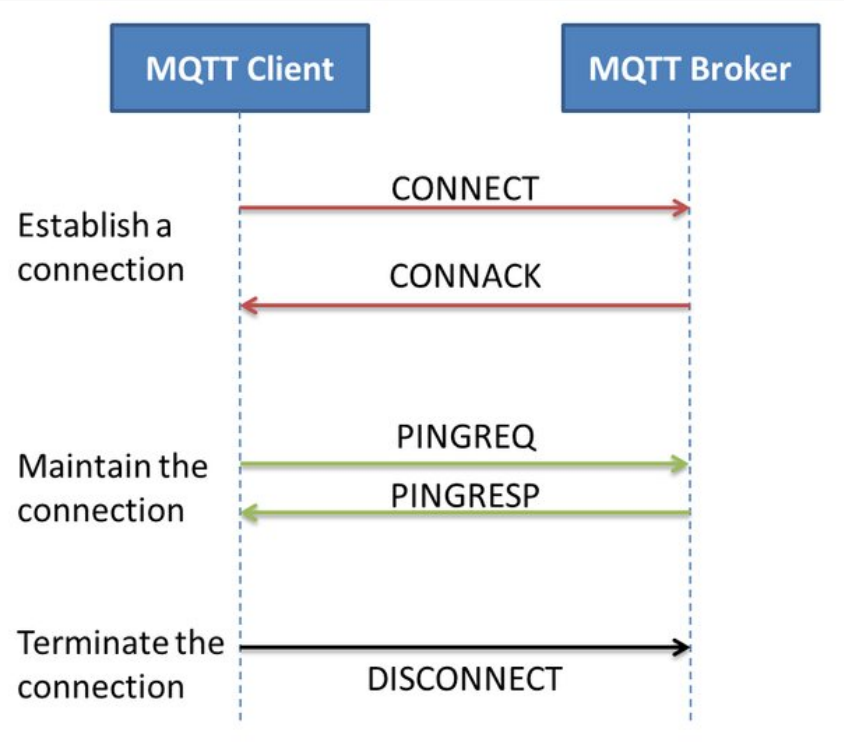
- Protocolo diseñado por IBM en 1999 para la comunicación con dispositivos de recursos limitados
- Estándar OASIS abierto y recomendación ISO (ISO/IEC 20922)
- Sobre TCP (puerto 1883) o TLS/TCP (puerto 8883)
  - También sobre UDP: MQTT for Sensor Networks (MQTT-SN)
- Mensajes MQTT codificados como secuencias “binarias”
  - Mensajes pequeños que caben en una única trama del nivel de enlace (ideal para enviar sobre 6LoWPAN/IEEE 802.15.4)
- Soporte para escenarios petición/respuesta (MQTT v5)

MQTT: Formato de mensaje

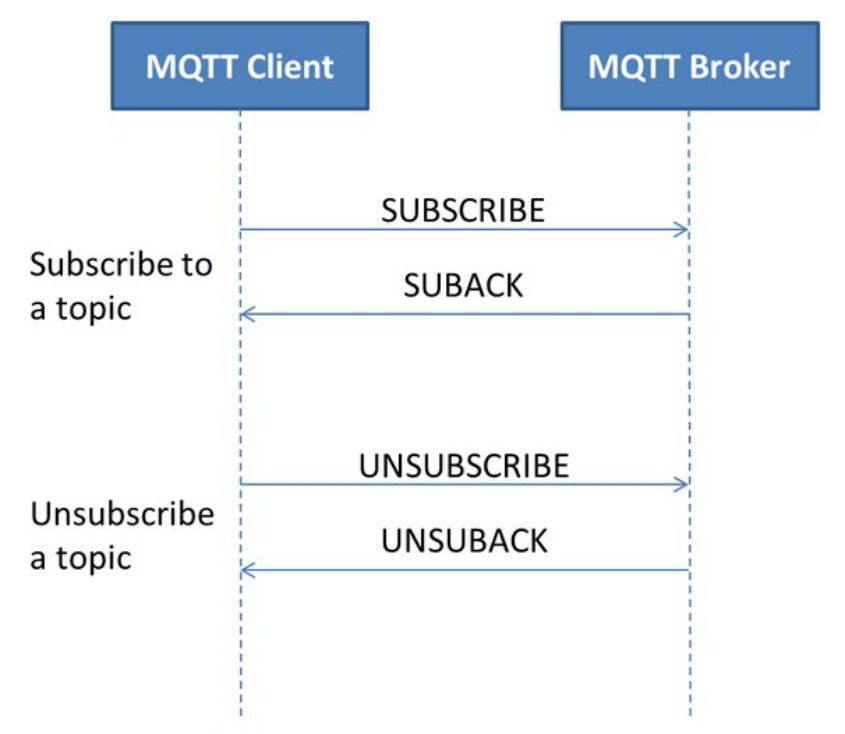


- Cabecera fija (2–5 bytes)
  - Tipo de mensaje (4 bits): CONNECT, PUBLISH, SUBSCRIBE...
  - *Flags*: duplicado, QoS (2 bits) y retención
  - Longitud (1–4 bytes): tamaño de la cabecera variable y los datos
    - Entero de bytes variables: el bit más significativo de cada byte es un indicador de “continuación”
- Cabecera variable + datos (*payload*)

MQTT: Conexión/suscripción

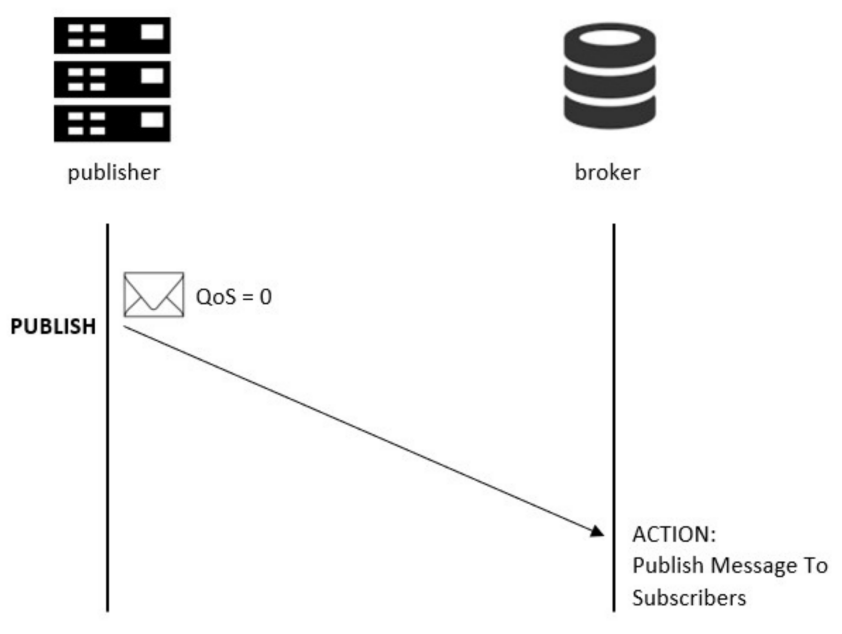


Conexión al *broker*



Suscripción a un *topic*

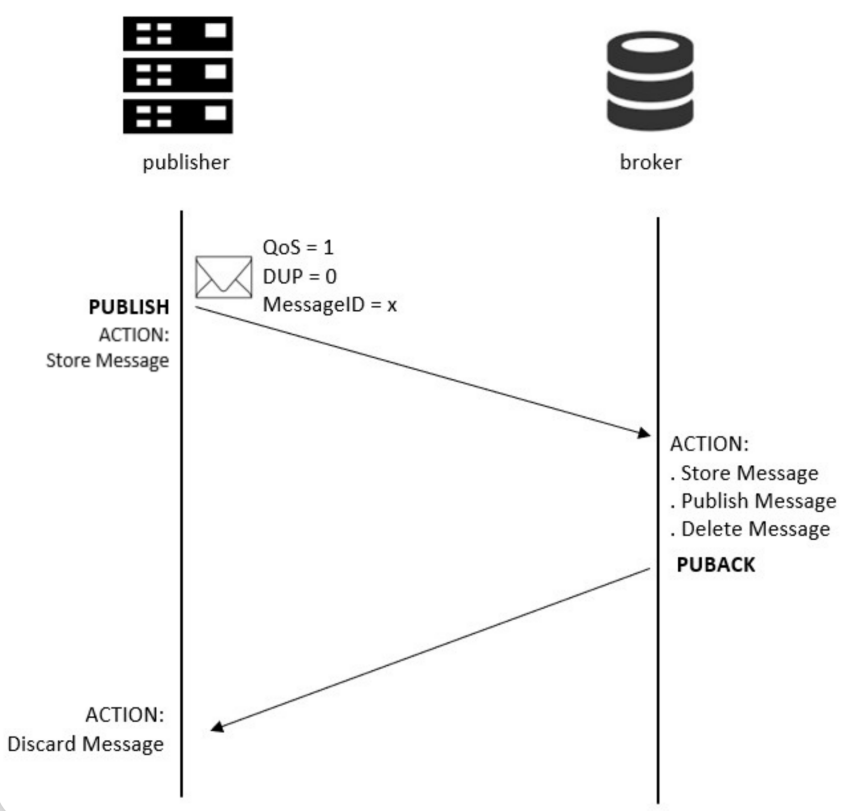
MQTT: Publicación *at most once*



At most once:

- Nivel QoS 0
- Envío *fire-and-forget*
  - El *broker* no confirma la publicación
- El publicador confía en TCP para garantizar la entrega
  - Pero la conexión TCP podría caerse...

MQTT: Publicación *at least once*



At least once:

- Nivel QoS 1
- El *broker* envía mensaje de confirmación
  - Si no recibe confirmación, el publicador reenvía el mensaje
  - Gestión de duplicados: *packet* ID y DUP=1

MQTT: Publicación *at least once*



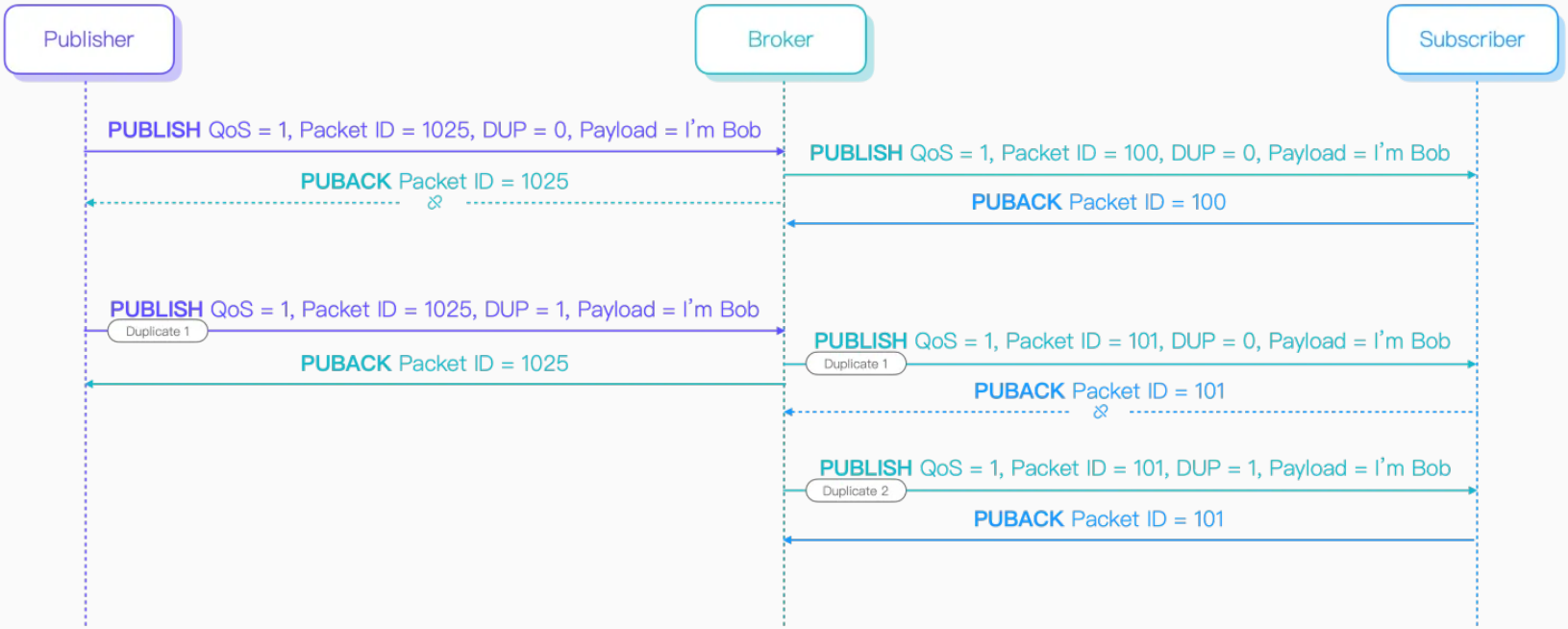
- En el caso 2, aunque DUP=1, el *broker* no puede asumir que ya ha recibido este mensaje y debe tratarlo como si fuese nuevo

MQTT: Publicación *at least once*



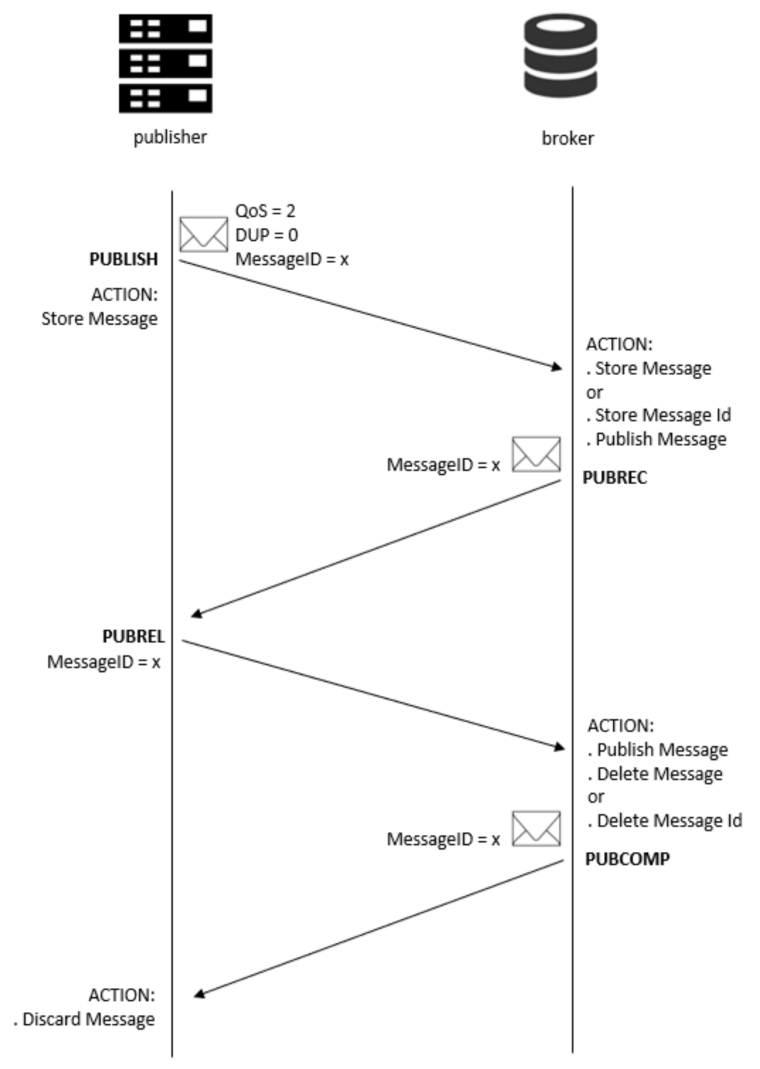
- En el caso 1, el segundo PUBLISH es un duplicado
- En el caso 2, se reutiliza el identificador para un nuevo mensaje
- Como es imposible distinguir entre ambos escenarios, todos los PUBLISH con DUP=1 deben tratarse como mensajes nuevos

MQTT: Publicación *at least once*



- Los suscriptores pueden recibir mensajes duplicados

MQTT: Publicación *exactly once*



Exactly once:

- Nivel QoS 2
- Evita que los suscriptores reciban mensajes duplicados
- Publicación en dos pasos
  1. Confirmación de la recepción del PUBLISH
  2. Confirmación de la publicación del mensaje
- Mayor sobrecarga y latencia

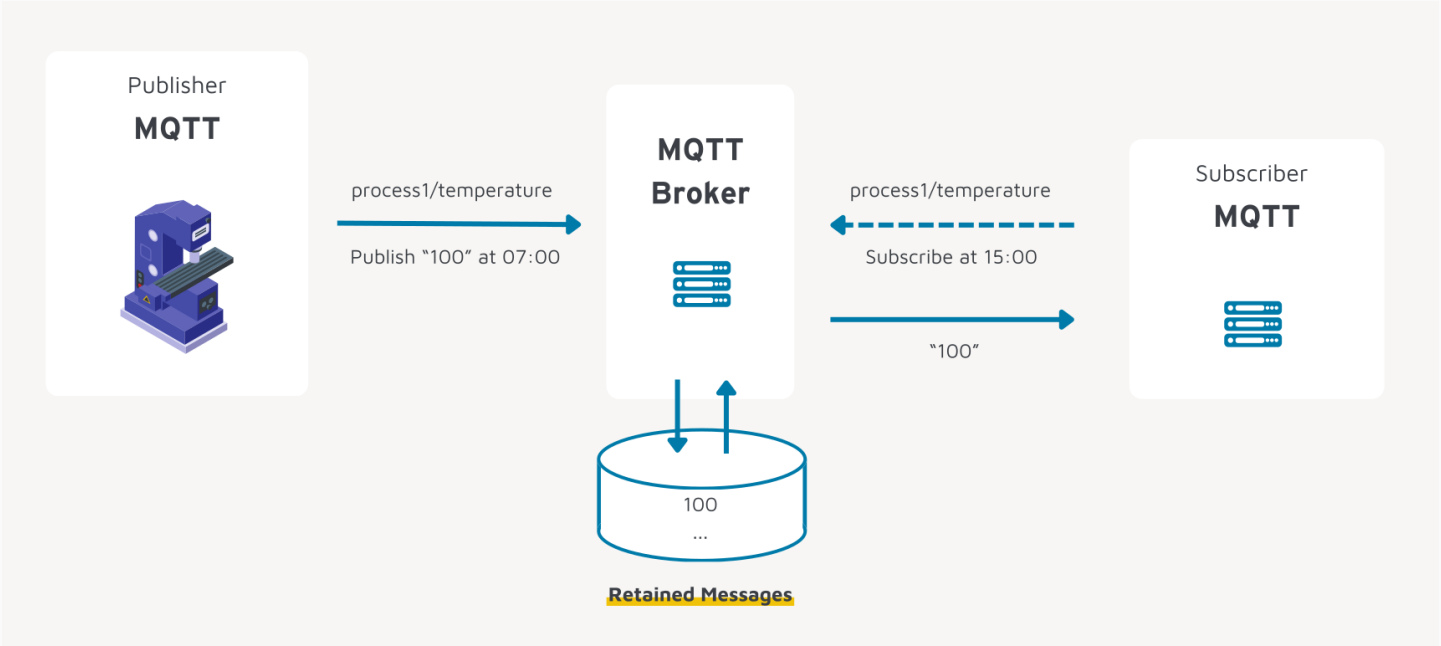
MQTT: Mensajes retenidos

Mensajes retenidos

Mensajes que el *broker* almacena para enviarlos de manera automática a los nuevos suscriptores de un *topic*

- El *broker* guarda el último mensaje publicado en un *topic* con el *flag* de retención activo
  - Cualquier mensaje retenido nuevo sobrescribe al anterior
- Cuando un nuevo cliente se suscribe al *topic*, el *broker* le envía inmediatamente el último mensaje guardado
  - Elimina la necesidad de esperar a que el dispositivo publicador envíe una nueva actualización
- Sirven para conocer el estado actual de un dispositivo de forma instantánea sin tener que esperar a que vuelva a transmitir

MQTT: Mensajes retenidos



- Los mensajes retenidos se entregan a los nuevos suscriptores también con el *flag* de retención activo
  - Les permite saber que están recibiendo un dato histórico o de configuración inicial, y no un evento en tiempo real

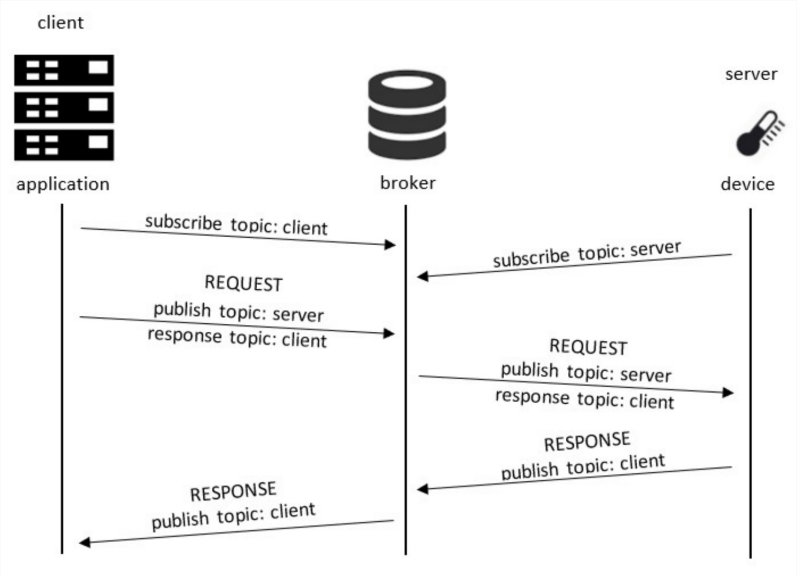
MQTT: *Last Will and Testament*

Last Will and Testament (LWT)

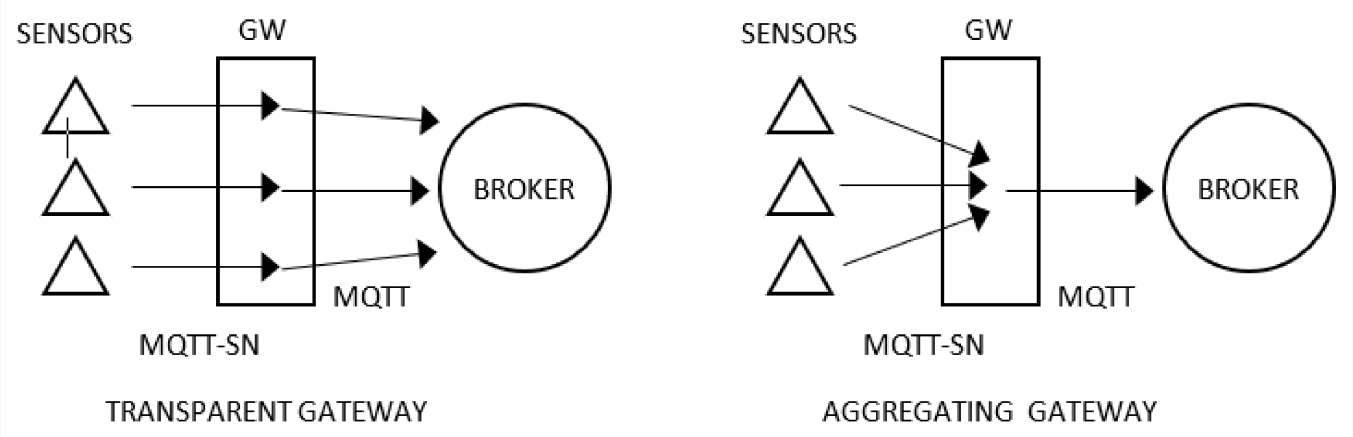
Función que permite a un cliente registrar un mensaje de "testamento" al conectarse en el *broker*

- El cliente indica en el mensaje CONNECT el *topic* del LWT, su contenido, su nivel QoS y si debe retenerse
- El *broker* guarda este mensaje pero no lo publica mientras la conexión con el cliente siga activa
- El *broker* publicará este mensaje automáticamente si el cliente se desconecta de forma inesperada
- Si el cliente se desconecta voluntariamente (envío mensaje DISCONNECT), el *broker* elimina el LWT sin publicarlo
- Sirve principalmente para que los demás dispositivos sepan de inmediato si alguno de ellos se ha quedado sin conexión

MQTT: Soporte petición/respuesta



- El cliente se suscribe a un *topic* propio privado (*response topic*)
- El servidor se suscribe a un *topic* público conocido
- El cliente envía la petición publicando en el *topic* del servidor
  - Incluye el *response topic* al que enviar la respuesta y un identificador único (*correlation data*)
- El servidor envía la respuesta publicando en el *response topic*
  - Incluye el mismo *correlation data* que la petición original



MQTT for Sensor Networks (MQTT-SN)

- MQTT sobre UDP
- Nuevo formato de mensajes para reducir su tamaño
  - *Gateway* para traducir mensajes MQTT-SN a mensajes MQTT
  - Transparente: cada mensaje MQTT-SN genera un mensaje MQTT
  - Agregador: múltiples mensajes MQTT-SN se agregan en un único mensaje MQTT